

JavaScript Done Right

Give your site some web 2.0-ness with jQuery

Nimish Chandiramani

readersletters@thinkdigit.com

We've all wished that our web pages would have some really cool animations, but we've all thrown those wishes out the window as soon as we discovered how tedious they are to do in JavaScript. It's clunky and cumbersome, and not designer-friendly at all. That's why a really smart egg named John Resig came up with jQuery, which is a “fast and concise JavaScript Library that simplifies HTML document traversing, event handling, animating, and AJAX interactions for rapid web development.” Basically, Resig's done all the dirty work, and all you have to do is call the functions he's written.

To use jQuery, you need to know your way around CSS, and this tutorial will assume that you do. You'll also need some basic knowledge of JavaScript, but any experience with a C-like language will help you understand the code.

The First jQuery

To include the jQuery framework in HTML, download the latest file from jquery.com, or refer the online version:

```
<script type="text/javascript" src=" HYPERLINK
"http://jquery.com/src/jquery-latest.js" http://
jquery.com/src/jquery-latest.js"/>
```

This ensures you've always got the latest jQuery running, so you don't have to worry about upgrading. On the other hand, if the latest version decides to drop a function that you're using, things may go horribly wrong.

Once you've included the jQuery framework, you can start accessing its functions. Let's start with initialising stuff on the page:

```
$(document).ready(function () {
    //put your initialisation code
    here
});
```

The “\$” is the “query” in jQuery - you use it to select elements in your document. You use the standard CSS selectors here: `$( 'div' )`, for example, will select all divs in the document, `$( '.someclass' )` will select all elements of the class “someclass”, and so on.

The next bit - `.ready( function () { //function code } );` - attaches the function to the document's “ready” event. In plain English, when the document ready, the function will execute. Let's start with a simple example:

```
//whitespaces aren't bothersome, so you can “pretty-
print” code if you want
$(document).ready(
(
    function()
    {
        $( 'a.alert' ).click
        (
            function()
            {
                alert( 'Hello!' );
            }
        );
    }
);
```

);

and here's what the HTML code looks like:

```
<a class="alert" href="#">Click here for a
```



The jQuery version of a “Hello World” script

```
message!</a>
```

and here's the result:

And now, we can move to bigger, better things—like messing about with design.

The jQuery Style

With jQuery, you can manipulate elements' CSS properties to no end—add classes, remove classes, alter styles, and so on. For example, let's say that you've got 30 divs—all of different classes—and you want one class to distinguish itself by turning bright yellow at the click of a button. That's when you use jQuery to change the CSS code:

```
$(document).ready(function() {
    $( "#buttonID" ).click(function() {
//select the button by its ID
//select oldclass divs
and change the background
$( "div.oldclass" ).
css({background-color});
});
```

As you can see, we've used the CSS-style selector “#buttonID” to select the button, and then attached a function to its “click” event. You can use these selectors to alter the layout of the document as well:

```
$(document).ready( function() {
    $( '#mergeButton' ).click(
function() {
        $( '#blue' ).
appendTo( '#green' ).parent().appendTo( '#red' );
    });
});
```

The “appendTo()” function, true to its name, appends the blue div to the green. The “parent()” then selects the green div (which is now the parent of the blue), and finally, the green div gets appended to the red. The HTML code here goes from this:

```
<div id="red"></div>
<div id="green"></div>
<div id="blue"></div>
to this:
<div id="red">
    <div id="green">
        <div id="blue"></div>
    </div>
</div>
```